SOFTWARE TEST PLAN

1. Test Objectives

The purpose of testing is to show that TrackWIL works correctly, meets the requirements in the Software Requirements Specification (SRS) and is ready for the live demonstration. Testing must demonstrate that:

- students, coordinators and lecturers can sign in and are sent to the correct dashboard, and invalid sign-ins are refused (FR-01 to FR-03);
- students can create entries, attach evidence and submit them for verification (FR-04 to FR-06);
- workplace mentors can approve or reject an entry through a one-time link without an account, and used, expired or unknown links are refused (FR-07, FR-08);
- statuses, verified hours and the monitoring dashboard are correct (FR-09 to FR-11);
- timestamps, evidence protection and sign-out work (FR-12 to FR-14);
- the non-functional requirements for performance, security, integrity, maintainability, compatibility and privacy are met (NFR-02 to NFR-07), and usability is measured when permission allows (NFR-01);
- defects are recorded, fixed and retested before the final submission.

2. Scope

Included: every functional requirement FR-01 to FR-14 and every non-functional requirement NFR-01 to NFR-07 in the SRS, tested through the user interface, the Server Actions and the database.

Excluded: load testing with many simultaneous users; formal penetration testing by an external party; browsers other than current Chrome and Edge; features deferred to future versions (mentor accounts, written feedback, notifications, reports); testing with real personal data.

3. Test Environment

Table 1: Test environment

Item	Details
Hardware	Windows laptop used for development
Operating system	Microsoft Windows 11 Pro
Runtime	Node.js 24.21
Application	TrackWIL production build (npm run build, then npm start)
Database	SQLite (data/trackwil.db), reset to the synthetic seed data with npm run db:reset before each run
Browsers	Google Chrome 155.0 and Microsoft Edge 155.0, at 1280 px and 375 px wide
Test tools	Manual testing in the browser; automated system tests (tests/system-tests.mjs, npm run test:system); browser DevTools device mode

Item	Details
Test data	Synthetic accounts: student1 and student2@ctucareer.co.za (student123), coordinator@ctucareer.co.za (coord123), lecturer@ctucareer.co.za (lecturer123); seeded entries in every status and one expired link

4. Test Types

Table 2: Test types used

Type	Purpose	Test cases
Functional (manual)	Check the main user journeys through the interface	TC-001 to TC-014
System (automated)	Check every functional requirement end to end, including the database	TC-110 to TC-141
Security	Access control, forged cookies, evidence protection, password storage, link misuse	TC-117 to TC-121, TC-126, TC-133 to TC-136, TC-142
Validation	Required fields, hour limits, file types, password and email rules	TC-012, TC-114 to TC-116, TC-123, TC-125, TC-128
Integrity	Database constraints and transactions	TC-136, TC-143
Performance	Response times against the 2-second target	TC-144
Compatibility	Chrome and Edge, phone and desktop widths	TC-145, TC-146
Maintainability and privacy	Type checking, linting, README, stored data	TC-147, TC-148
Usability	Timed task with first-time students (with consent)	TC-149
Regression	Rerun the full automated suite after every fix	All automated tests

5. Entry and Exit Criteria

Testing can begin when:

- the features to be tested are merged into a branch and the production build succeeds;
- the database has been reset to the seed data;
- test accounts and test data are available;
- the test cases, with their expected results, are written in the Test Cases and Results workbook.

Testing is complete when:

- every planned test case has been run, or its Blocked status is explained;

- actual results and Pass, Fail or Blocked statuses are recorded with the tester, date and evidence;
- every defect is in the Defect Log; all Critical and High defects that affect users are fixed and retested;
- the full automated suite passes on the final build;
- the Requirements Traceability Matrix links every requirement to its test cases and result;
- the results are summarised in Chapter 5.

6. Roles and Responsibilities

Table 3: Testing roles

Role	Member	Responsibilities
Test lead and QA	Dylan Shepard	Reviews test cases and results; checks that requirements are covered; confirms the exit criteria
Manual tester	Sihle Kayise	Runs and records the manual test cases (TC-001 to TC-014); captures screenshots
Automated testing and fixes	Gedion Basson	Writes and runs the automated system tests; fixes and retests defects; keeps the workbooks up to date
Developer	Moegammad Green	Investigates and fixes assigned defects; supports retesting
Whole team	All members	Reviews the results and agrees whether the system is ready for the demonstration

7. Defect Management

1. **Record:** each defect gets an ID (DEF-xx), the date, the feature, a description with steps to reproduce, the severity (Low, Medium, High or Critical) and the priority.
2. **Reproduce:** the defect is confirmed on the affected build before any fix is made.
3. **Assign:** the defect is assigned to a member, who fixes it on a branch named name/task.
4. **Fix:** the fix is committed with a message that names the defect and is merged through a pull request.
5. **Retest:** the original test case is run again and the full automated suite is rerun (regression).
6. **Close or reopen:** if the retest passes, the defect is closed with the commit and test-case reference; otherwise it is reopened.

8. Schedule

Table 4: Test schedule

Date	Activity
22 October 2026	Manual functional testing (TC-001 to TC-014)
22 October 2026	Database migration verified with 28 automated checks
23 October 2026	Automated system tests, security review, browser and mobile checks; DEF-01 to DEF-03 fixed and retested
24 to 27 October 2026	DEF-04 and DEF-05 fixes; usability test if permission is granted; regression run
28 October 2026	Final regression run on the submission build